

Portal web CAAN

UTN Mar del Plata · Gestión del Desarrollo de Software
Stutz, Tomás · Sánchez, Kevin · Sollberger, Tomás | Prof. Fernández Carbonell, César Augusto

React + Node.js + MongoDB

Scrum · 3 Sprints

ONG sin fines de lucro

Presupuesto \$0

PROBLEMA → SOLUCIÓN

Problema	Solución
El CAAN tiene su información dispersa en redes sociales. Ante emergencias animales, la comunidad no sabe a qué canal acudir ni qué protocolos seguir.	Plataforma web autogestionable que centraliza el catálogo de adopciones, noticias, donaciones, FAQ y un panel de administración seguro para el staff.

MÉTRICAS DEL PROYECTO

66 SP	21 SP/s print	6 semanas	\$1.17M ARS	\$0 ARS	3 devs
Esfuerzo total	Velocidad estimada	Duración	Costo de desarrollo	Costo tecnológico	Integrantes

ALCANCE FUNCIONAL

Incluido	Excluido
Catálogo público de adopciones con filtros · Panel admin CRUD animales y noticias · Autenticación JWT segura · Módulo de donaciones y socios · Sección noticias/campañas · FAQ · Editor enriquecido BlockNote · Carga de imágenes a Cloudinary	Pasarela de pagos integrada · Formulario de denuncias formales · Chat en tiempo real · App móvil nativa iOS/Android

STACK TECNOLÓGICO

React v19	Node.js + Express v5	MongoDB Atlas	JWT + httpOnly	Cloudinary	Tailwind CSS v4	Zod validaciones	BlockNote Editor	Vercel + Render
-----------	----------------------	---------------	----------------	------------	-----------------	------------------	------------------	-----------------

PLANIFICACIÓN · 3 SPRINTS (SCRUM)

Sprint 1	Cimientos de arquitectura y persistencia core Monorepo con pnpm · Conexion MongoDB Atlas · Esquemas Zod · CRUD de animales en backend
Sprint 2	Capa de seguridad e interfaz del adoptante Autenticacion JWT dual con cookies httpOnly · Catalogo publico con filtros dinamicos · Diseno Mobile-First
Sprint 3	Contenidos, testing y despliegue Modulo de noticias + BlockNote · Dashboard admin completo · Fallback Cloudinary (Chain of Responsibility) · E2E testing · Deploy en Vercel + Render

GESTION DE RIESGOS

R 1	Saturacion de cuota en Cloudinary	Medio
R 2	Indisponibilidad de un miembro del equipo	Medio
R 3	Retrasos por complejidad del despliegue	Alto
R 4	Acoplamiento excesivo en controladores del backend	Medio
R 5	Filtracion de informacion sensible por errores del servidor	Alto
R 6	Abandono del sistema por el personal del refugio	Medio

DECISIONES TECNICAS CLAVE

Seguridad Tokens JWT en cookies httpOnly/Secure para mitigar XSS. Contraseñas encriptadas con bcrypt. Validaciones estrictas con Zod en todos los endpoints.	Confiabilidad multimedia Patron Chain of Responsibility con hasta 3 cuentas Cloudinary en cascada para evitar caídas por saturación de cuota gratuita.
Arquitectura backend Separación en capas: Rutas, Middlewares, Controladores, Service Layer y Modelos. Principio de responsabilidad única (SRP).	Testing 4 niveles: unitario, integración, E2E y aceptación con la ONG. Feature Freeze ante bugs recurrentes para priorizar calidad sobre velocidad.